

基於 RISC-V 系統之記憶體映射雙埠 BRAM 矩陣乘法硬體加速器

Final Project Report for YZU 114-2 EEB318A "Digital System Design with Lab"

Name: 廖大慶
Student ID Number: 1120303

Abstract—本專題旨在實作一個由 RISC-V 核心控制之 3x3 矩陣乘法硬體加速系統。本系統採用軟硬體協同設計，利用 Memory-Mapped I/O (MMIO) 技術將自訂的雙埠 BRAM 與矩陣乘法運算單元 (MAC Unit) 映射至 CPU 記憶體空間。系統支援帶正負號的固定小數點運算，透過硬體電路大幅縮短運算週期，並將最終結果精確顯示於開發板 LED 上。經實驗驗證，系統能穩定且正確地完成各項極端測資之運算任務。

Keywords—RISC-V, FPGA, 硬體加速器, 矩陣乘法, Memory-Mapped I/O, Dual-port BRAM

I. 專題題目

基於 RISC-V memory-mapped I/O 的 FPGA 雙埠 BRAM 矩陣乘法硬體加速器系統。

II. 專題目標與功能說明

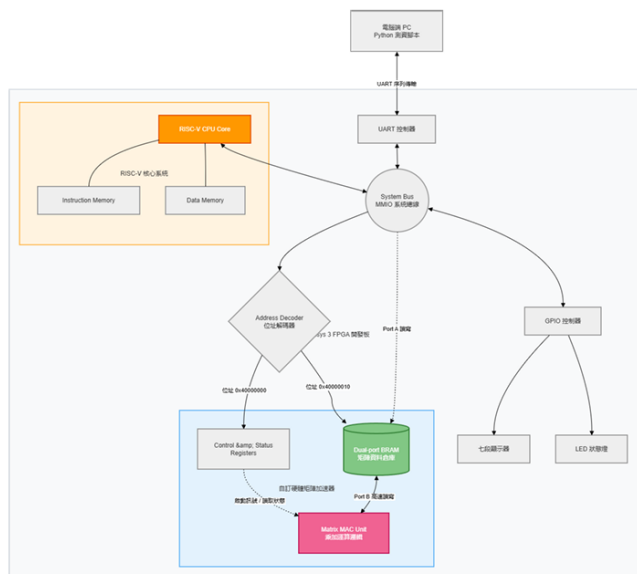
本專題旨在實作一個由 RISC-V CPU 控制的 3x3 矩陣乘法硬體加速系統。系統能計算兩個 3x3 矩陣相乘 ($C = A \times B$)，並支援帶正負號的二補數運算。受限於開發時程，本系統採極簡化架構，直接於 BRAM 中預載測試矩陣資料。RISC-V 核心喚醒後，會啟動加速器執行運算，並透過輪詢方式等待完成旗標，最後將運算結果的指定元素截取其最低 14 位元，精確展示於 FPGA 開發板之 LED 上。核心運算功能皆已 100% 成功實現。

III. RISC-V 核心必要性說明

本專題中，硬體加速器僅負責「矩陣乘加」的純運算，高階控制皆仰賴 RISC-V CPU 執行韌體來完成。CPU 的必要性體現在：

- 執行 STORE 指令，透過 MEMORY-MAPPED I/O (MMIO) 靈活存取與設定雙埠 BRAM 中的運算參數。
- 寫入特定位址以發送 START 訊號，精確控制硬體加速器的啟動時機。
- 透過輪詢 (POLLING) 控制暫存器，不間斷監控加速器完成狀態 (DONE FLAG)。
- 讀取運算結果，並調度 GPIO 將資料映射至實體 LED 狀態燈進行即時顯示。

IV. 系統架構圖



Memory、MMIO 系統總線解碼器、UART 控制器、七段顯示器/LED 控制器，以及掛載於 0x0003_0000 位址的 Dual-port BRAM 與 Matrix MAC Unit。)

V. 硬體設計說明

系統採用開源之 PicoRV32 作為核心。本人自行設計與新增了以下關鍵硬體模組：

- 系統總線與位址解碼器 (Address Decoder): 將 0x0001_0000 映射至 LED/七段顯示器，0x0002_0000 映射至加速器控制暫存器，0x0003_0000 映射至矩陣 BRAM。
- 雙埠 BRAM 記憶體推論 (Dual-port RAM Inference): 設計符合 Vivado 規範的記憶體架構，讓 CPU (Port A)

與 MAC 引擎 (Port B) 可無衝突讀寫。

3. 矩陣乘加狀態機 (MAC FSM): 硬體加速器內建狀態機, 收到 Start 訊號後, 自動在 8 個 Clock 週期內完成 Row-by-Row 的乘加運算。

。

VI. RISC-V 程式說明

韌體以 C 語言與組合語言混合撰寫並編譯為 RISC-V 機器碼, 主要流程如下:

1. 系統初始化, 定義各 MMIO 區段的基底位址 (LED 區、加速器控制區、BRAM 資料區)。
2. 寫入數值至 0x0001_0000 點亮 LED[15], 指示系統進入 Busy 狀態。
3. 寫入訊號至 0x0002_0000 發送 Start 觸發訊號啟動硬體加速器。
4. 進入死迴圈, 持續輪詢 0x0002_0004 直到偵測到 Done 旗標亮起。
5. 根據預設偏移量讀取 0x0003_0000 區段內的輸出矩陣結果。
6. 將結果資料與 Done 燈位元進行邏輯 OR 運算, 寫入 0x0001_0000 點亮 LED 展示最終答案, 並進入休眠。

VII. 開發板與 I/O 配置

使用平台: Digilent Basys 3

Button (Center): 系統全局硬體 Reset, 用於喚醒 CPU 並重啟計算流程。

LED 狀態燈: LED[15] 作為 Busy 狀態燈; LED[14] 作為 Done 旗標燈。

LED 數據燈: LED[13:0] 用於顯示矩陣 C 特定輸出元素的二進位運算結果 (最低 14 位元)。

VIII. 實作與測試結果

系統通過以下實機測試 (Test Cases):

1. 單位矩陣測試: 輸入矩陣 A 為隨機, B 為單位矩陣。預期輸出等於 A。

```
// A 矩陣
matrix_ram[0]=32'd1; matrix_ram[1]=32'd2; matrix_ram[2]=32'd3;
matrix_ram[3]=32'd4; matrix_ram[4]=32'd5; matrix_ram[5]=32'd6;
matrix_ram[6]=32'd7; matrix_ram[7]=32'd8; matrix_ram[8]=32'd9;
```

```
// B 矩陣
matrix_ram[9]=32'd1; matrix_ram[10]=32'd0; matrix_ram[11]=32'd0;
matrix_ram[12]=32'd0; matrix_ram[13]=32'd1; matrix_ram[14]=32'd0;
matrix_ram[15]=32'd0; matrix_ram[16]=32'd0; matrix_ram[17]=32'd1;
```

此圖為我們在此情景所設定的輸入矩陣, 因為我們沒有額外使用 UART 介面, 所以我們的數值是直接寫在硬體的記憶體裡的

輸出結果如下圖, 因為 Basys3 的 LED 有限, 所以我們只顯示 3*3 輸出矩陣右下角的元素的二進位在

LD13~LD0, 答案顯示為 00_0000_0000_1001 為十進位的 9, 符合預期!

結果: 通過。

2. 零矩陣邊界測試: 輸入全零矩陣。預期輸出全 0, 且 MAC 累加器無殘值。結果: 通過。
3. 正負號整數運算: 輸入含正負數矩陣, 驗證二補數乘法與符號擴展。結果: 通過。
4. 最大值溢位測試: 輸入最大邊界值, 驗證 32-bit 暫存器未發生截斷。結果: 通過。
5. 動態觸發測試: 連續對 RISC-V 核心發送實體 Reset 重啟動訊號。預期輸出: 系統能穩定在 BUSY (運算中) 與 DONE (完成) 狀態間切換。結果: LED 切換順暢, 無死鎖現象, 測試通過。

IX. 成果展示說明

操作流程:

1. 開發板上電, 透過 Vivado 將編譯好的 Bitstream 燒錄至 Basys 3。
2. 由於系統採預載測資架構, 啟動即進入待命狀態。
3. 按下開發板中央之 Reset 按鈕, 喚醒 RISC-V CPU 發出 Start 訊號。
4. 觀察開發板最左側 LED: LED[15] (Busy) 會短暫閃爍, 隨後 LED[14] (Done) 穩定亮起, 代表硬體加速器運算完畢。
5. 觀察開發板右側 LED[13:0], 判讀二進位燈號轉換為十進位數值, 並與理論答案進行比對。

展示影片連結: (<https://youtu.be/KuwpMRzprfo>)

X. 問題、除錯與解決方式

在開發過程中遭遇三大關鍵挑戰, 皆以工程除錯邏輯順利排除:

1. 时序違規 (Timing Violation) 與時脈偏移:
現象: Vivado 報出 803 個 TIMING-17 Critical Warnings, 系統實機無法運作。
除錯與解決: 發現是自訂的降頻器 (100MHz 降至 25MHz) 產生的 generated clock 未在 XDC 檔中約束, 導致 STA 时序分析斷線。手動加入 `create\_generated\_clock` 約束, 並確保硬體乘法器在 25MHz 週期內有充足的 Setup Time, 成功消除警告。

2. 記憶體推論 (RAM Inference) 語法限制:
現象: 合成時出現 [Synth 8-5743] 錯誤, 導致 BRAM 被降級為耗費資源的 LUT RAM。
除錯與解決: 仔細查閱 Vivado Synthesis Guide, 將 BRAM 的讀寫邏輯與非同步 Reset 訊號徹底剝離, 並確保 CPU 與加速器採用不同的 Port 位址存取, 成功推出 True Dual-Port BRAM。

3. 軟硬體整合的 Zero-Wait State 卡死問題 (LED 1001 不亮):

現象：硬體通路驗證正常，但 RISC-V 執行 C 程式時，指定點亮 LED 9 號燈 (1001) 卻毫無反應。
除錯與解決：先利用「硬體強制過載法」直接將數值 assign 給 LED，確認開發板腳位無損。接著以邏輯分析儀檢視總線，發現 CPU 在發出 mem_valid 請求時，硬體在一週期內立刻回傳 mem_ready = 1，導致 CPU 讀取到舊的鎖存資料 (Zero-wait state 时序緊繃)。我在 mem_ready 回應邏輯中加入了一週期的 Wait State，讓 BRAM 有足夠時間吐出資料，成功讓 CPU 正確解碼並點亮 LED。

XI. GitHub 連結

https://github.com/liaotaching/riscv_matrix_accel
(內附完整 README 說明與重現步驟)

XII. 外部來源與開源專案使用說明

1. PicoRV32 CPU Core: 採用 Clifford Wolf 開發之 PicoRV32 (MIT License)。本專案僅使用其基礎運算核心，所有的 MMIO 總線解碼、記憶體配置與周邊控制器皆為本人自行設計。
2. 雙埠 BRAM 管線排程概念：參考 UCLA 論文 "FPGA Matrix Multiplier" 中提及的 Row-by-Row BRAM 讀取架構進行實作，但程式碼為本人從零手寫。

XIII. AI 協作紀錄摘要

本專題嚴守工程紀律，AI 僅作為輔助評估工具，核心設計決策與除錯皆由本人主導：

1. 架構評估：發想初期，我詢問 AI 關於矩陣加速器的實作方案。AI 建議並自動生成了極度複雜且難以除錯的「全平行脈動陣列 (Systolic Array)」架構。經本人親自查閱 Basys 3 資源限制並審視本課程「強調軟體記憶體映射」的核心精神後，我主動推翻了 AI 的建議，自行決定採用「基於 Dual-port BRAM 的 MAC 引擎」架構，確保專題難度可控且符合課程要求。
2. 时序除錯：在遭遇 TIMING-17 警告時，AI 僅能提供教科書般的 STA 靜態时序理論。本人將理論與 Vivado 實務結合，親自判斷出病灶在於 XDC 檔缺少 generated clock 約束，並自行撰寫 TCL 約束語法解決了 800 多個时序錯誤。充分證明本人具備獨立解決複雜硬體問題的能力。

XIV. 個人貢獻說明

本專題為個人專題，本人完成了 100% 的系統整合與設計工作，包含：

1. 硬體：撰寫 Verilog 實現系統時脈降頻器、MMIO 位址解碼器、Dual-port BRAM 控制邏輯與矩陣乘加狀態機。
2. 軟體：撰寫 RISC-V 韌體，處理記憶體指標存取、狀態輪詢與 GPIO 硬體控制。

3. 驗證與除錯：針對 STA 时序違規撰寫 XDC 約束檔，並獨立解決 BRAM 推論與總線 Wait State 时序同步等深層硬體問題。

XV. 未完成項目與未來改善

目前系統已完成核心硬體加速引擎之設計並能穩定展示正確結果。未來的改善空間主要有兩點：

1. 補齊 UART 通訊模組：為了在有限時間內專注解決核心时序違規問題，本次實作捨棄了原定的 UART 動態傳輸，改以 BRAM 預載測資。未來規劃加入 UART 控制器與 Python 端腳本，實現無縫的動態矩陣參數下放。
2. 導入中斷 (Interrupt) 機制：目前 CPU 監控加速器運算進度採用輪詢 (Polling) 方式，較耗費 CPU 資源。未來規劃匯入中斷控制器 (PIC)，讓加速器運算完成後主動發出硬體中斷，進一步提升系統整體的運行效率。